

Factorisation de Bunch-Parlett et Bunch-Kaufman

MTH8211 - Projet phase 2

Hortense Beaudoin et Téo Dumoutier

Section 1 . Description du projet

Section 2 . Théorie

Section 3 . Implémentation

Section 4 . Tests et performance

Section 5 . Discussion

Section 6 . Retour sur les objectifs

1 Description du projet

Si on tente de résoudre un système $Ax = b$ où A est dense, plusieurs solutions existent, mais une majorité sont très coûteuses. La décomposition QR et la SVD existent pour toute matrice A , y compris les matrices indéfinies (i.e il existe x t.q $x^T Ax \leq 0$ et il existe y t.q $y^T Ay \geq 0$), mais elles n'exploitent pas forcément la structure du système à résoudre.

Certaines structures symétriques sont fréquemment rencontrées lors de la résolution de problèmes. On a par exemple vu en classe qu'une autre manière d'exprimer le problème de moindres carrés est de résoudre plutôt le système de point de selle

$$\begin{bmatrix} I & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} r \\ x \end{bmatrix} = \begin{bmatrix} b \\ 0 \end{bmatrix}$$

où la matrice formée de A et A^T est symétrique et toujours indéfinie. Sous cette forme, le bloc supérieur gauche est bien évidemment inversible, puisqu'il est égal à l'identité. De manière intéressante, toute autre matrice B non nulle et symétrique peut être permutée pour adopter une forme où le bloc supérieur gauche est inversible, soit

$$PBP^T = \begin{bmatrix} E & C^T \\ C & D \end{bmatrix}$$

où $D = D^T$ et $E = E^T$ inversible. Ce type de problème est donc particulièrement fréquent et il est intéressant de s'intéresser à une méthode de résolution où la symétrie des matrices est exploitée.

La décomposition de Cholesky n'existe pas pour ce genre de système puisque la matrice considérée n'est pas définie-positive, ce qui est une exigence associée à ce type de factorisation. Une factorisation LDL^* pourrait exister, mais pas systématiquement et elle court le risque d'être numériquement instable. Higham illustre bien ce risque dans son livre avec l'exemple suivant:

$$A = \begin{bmatrix} \epsilon & 1 \\ 1 & \epsilon \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1/\epsilon & 1 \end{bmatrix} \begin{bmatrix} \epsilon & 0 \\ 0 & \epsilon - 1/\epsilon \end{bmatrix} \begin{bmatrix} 1 & 1/\epsilon \\ 0 & 1 \end{bmatrix}$$

A est symétrique et si $\epsilon < 1$, indéfinie. On remarque que dans le cas où $\epsilon = 0$ la factorisation n'existe tout simplement et que le reste du temps, les facteurs $1/\epsilon$ risquent de grossir de manière disproportionnée, menant à un système mal conditionné ou à des erreurs numériques importantes.

Ce projet va donc plutôt s'intéresser aux factorisations de Bunch-Parlett et Bunch-Kaufman, qui exploitent toutes deux l'idée de permutation mentionnée précédemment. La mise en place algébrique étant la même dans les deux cas, nous allons dès maintenant la présenter brièvement.

2 Théorie

2.1 Factorisation en bloc pour matrices symétriques

Tel que mentionné ci-dessus, toute matrice $A \in \mathbb{C}^{n \times n}$ symétrique non-nulle peut être permutée de la manière suivante:

$$PAP^T = \begin{bmatrix} E & C^T \\ C & D \end{bmatrix}$$

où $E \in \mathbb{C}^{s \times s}$, $C \in \mathbb{C}^{n-s \times s}$ et $D \in \mathbb{C}^{(n-s) \times (n-s)}$ avec $s \in 1, 2$ et E inversible. Sous cette forme, il est possible de procéder à la factorisation suivante:

$$PAP^T = \begin{bmatrix} I_s & 0 \\ CE^{-1} & I_{n-s} \end{bmatrix} \begin{bmatrix} E & 0 \\ 0 & D - CE^{-1}C^T \end{bmatrix} \begin{bmatrix} I_s & E^{-1}C^T \\ 0 & I_{n-s} \end{bmatrix}$$

On reconnait $D - CE^{-1}C^T$ comme étant le complément de Schur de E dans la matrice PAP^T . Ce complément est aussi symétrique, par symétrie des blocs E et D ;

$$(D - CE^{-1}C^T)^T = D^T - CE^{-T}C^T = D - CE^{-1}C^T$$

On peut donc répéter le processus sur cette matrice de dimension $(n-s) \times (n-s)$, puis de nouveau et ainsi de suite jusqu'à l'obtention de la factorisation complète.

À titre d'illustration, imaginons être à l'itération k avec la forme suivante;

$$P_k A_k P_k^T = \begin{bmatrix} E_k & C_k^T \\ C_k & D_k \end{bmatrix} = \begin{bmatrix} I_{s_k} & 0 \\ C_k E_k^{-1} & I_{n-s_k} \end{bmatrix} \begin{bmatrix} E_k & 0 \\ 0 & D_k - C_k E_k^{-1} C_k^T \end{bmatrix} \begin{bmatrix} I_{s_k} & E_k^{-1} C_k^T \\ 0 & I_{n-s_k} \end{bmatrix}$$

On pose alors $A_{k+1} = D_k - C_k E_k^{-1} C_k^T$ et on procède de la même manière.

$$P_{k+1} A_{k+1} P_{k+1}^T = \begin{bmatrix} E_{k+1} & C_{k+1}^T \\ C_{k+1} & D_{k+1} \end{bmatrix} = \begin{bmatrix} I_{s_{k+1}} & 0 \\ C_{k+1} E_{k+1}^{-1} & I_{n-(s_k+s_{k+1})} \end{bmatrix} \begin{bmatrix} E_{k+1} & 0 \\ 0 & D_{k+1} - C_{k+1} E_{k+1}^{-1} C_{k+1}^T \end{bmatrix} \begin{bmatrix} I_{s_{k+1}} & E_{k+1}^{-1} C_{k+1}^T \\ 0 & I_{n-(s_k+s_{k+1})} \end{bmatrix}$$

Et ainsi de suite. Les matrices P , L et B sont construites de la façon suivante à chaque itération:

$$P_{k+1} = P_{k+1} P_k \dots P_1$$

$$L_{k+1} = \begin{bmatrix} I_{s_k} & 0 & 0 \\ (C_k E_k^{-1})[1 : s_{k+1}, 1 : s_{k+1}] & I_{s_{k+1}} & 0 \\ (C_k E_k^{-1})[s_{k+1} + 1 : n, 1 : s_{k+1}] & (C_{k+1} E_{k+1}^{-1}) & I_{n-(s_k+s_{k+1})} \end{bmatrix}$$

$$B_{k+1} = \begin{bmatrix} E_k & 0 & 0 \\ 0 & E_{k+1} & 0 \\ 0 & 0 & A_{k+2} \end{bmatrix}$$

où les P et les L alternent dans la construction (puisque les L subissent aussi la permutation.) La différence entre les processus de Bunch-Parlett et Bunch-Kaufman repose dans le choix du pivot, soit de la permutation P_k . Le tout sera exploré plus en détail dans les prochaines sections.

2.2 Bunch-Parlett

La stratégie de Bunch-Parlett pour le choix du pivot est aussi appelée *pivotage complet*. Elle se base sur les valeurs maximales des entrées de la matrice sur et hors diagonale. Le choix du pivot est effectué comme suit;

Poser $\alpha = (1 + \sqrt{17})/8$

Poser $\mu_0 = \max_{i,j} |a_{ij}| =: |a_{pq}|$, ($q \geq p$)

Poser $\mu_1 = \max_i |a_{ii}| =: |a_{rr}|$

Si $\mu_1 \geq \alpha \mu_0$

Utiliser a_{rr} comme pivot 1×1 ($s = 1$, les colonnes et rangées 1 et r sont permutées)

Sinon

Utiliser $\begin{bmatrix} a_{pp} & a_{qp} \\ a_{qp} & a_{qq} \end{bmatrix}$ comme pivot 2×2 ($s = 2$, les colonnes et rangées 1 et p sont permutées, suivies des colonnes et rangées 2 et q)

On peut visualiser l'emplacement des pivots comme suit;

$$\begin{pmatrix} \ddots & \dots & \dots & \dots & \dots & \dots & \dots \\ \vdots & a_{rr} & & & & & \\ \vdots & & \ddots & & & & \\ \vdots & & & a_{pp} & \dots & a_{qp} & \\ \vdots & & & \vdots & \ddots & \vdots & \\ \vdots & & & a_{qp} & \dots & a_{qq} & \ddots \\ \vdots & & & & & & \ddots \end{pmatrix}$$

Il est intéressant de regarder de plus près la valeur de α et la manière dont cette dernière est obtenue. Elle est dérivée directement de la borne sur les éléments des blocs de B , elle-même issue des valeurs maximales contenues dans A . Comme cette valeur est reprise dans les processus de Bunch-Kaufman ainsi que dans l'alternative du *rook pivoting*, nous trouvons pertinent d'en présenter la brève dérivation ici. À noter que la dérivation s'effectue de manière semblable pour les deux autres processus et mène au même résultat.

2.2.1 Dérivation de α

Dans le cas du pivot 1×1 , ce dernier est choisi si $\mu_1 \geq \alpha \mu_0$, i.e $\max_i |a_{ii}| \geq \alpha \max_{i,j} |a_{ij}|$. Toutes les mises à jour de A via le complément de Schur sont ainsi bornée;

$$|a_{ij}| = |b_{ij} - c_{i1} \frac{1}{e_{11}} c_{1j}| \leq \mu_0 + \frac{\mu_0^2}{\mu_1} \leq \left(1 + \frac{1}{\alpha}\right) \mu_0$$

On peut borner les valeurs de $|a_{ij}|$ de manière semblable pour le cas du pivot 2×2 . On note cette fois le complément de Schur comme

$$a_{ij} = b_{ij} - \begin{bmatrix} c_{i1} & c_{i2} \end{bmatrix} E^{-1} \begin{bmatrix} c_{11} \\ c_{21} \end{bmatrix}$$

où

$$E^{-1} = \frac{1}{\det(E)} \begin{bmatrix} e_{22} & -e_{12} \\ -e_{21} & e_{11} \end{bmatrix}$$

Par symétrie de E

$$\det(E) = e_{11}e_{22} - e_{21}^2 \leq \mu_1^2 - \mu_0^2 \leq (\alpha^2 - 1)\mu_0^2$$

Il est alors posé que $\alpha \in (0, 1)$, de telle sorte que $|\det(E)| \geq (1 - \alpha^2)\mu_0^2$. Il est alors possible de borner supérieurement E^{-1} dans le complément de Schur. En effet

$$|E^{-1}| \leq \frac{1}{(1 - \alpha^2)\mu_0^2} \begin{bmatrix} \alpha & 1 \\ 1 & \alpha \end{bmatrix}$$

et donc

$$|a_{ij}| \leq \mu_0 + \frac{1(1 + \alpha)\mu_0^2}{(1 - \alpha^2)\mu_0} = \left(1 + \frac{2}{1 - \alpha}\right) \mu_0$$

Finalement, pour obtenir la valeur de α , le multiplicateur de μ_0 dans la borne du pivot de 2×2 est rendu équivalent à deux fois le multiplicateur de μ_0 dans la borne du pivot de 1×1 , soit

$$\left(1 + \frac{1}{\alpha}\right)^2 = \left(1 + \frac{2}{1 - \alpha}\right)$$

Ce système peut être résolu et mène à deux valeurs de α , parmi lesquelles on choisit la valeur positive, qui est bien $\alpha = (1 + \sqrt{17})/8$.

Il s'agit d'une dérivation assez simple d'une valeur reprise dans chacune des implémentations auxquelles ce projet s'intéresse. Le concept de choix de pivot et de borne sur l'augmentation des éléments est aussi présent dans d'autres types de factorisation et dans des méthodes de base comme l'élimination de Gauss.

2.2.2 Pseudo-code

Pour l'implémentation, on combine le choix de p à la mise en place de base du problème, vue dans la première section du rapport. Le pseudo-code suivant illustre la stratégie d'implémentation *in-place* de cet algorithme, présentée dans la Section 3.

Algorithm 1: Factorisation de Bunch-Parlett (pivotage complet)

Result: $PAP^T = LBL^T$ **Input :** A hermitienne indéfinie**Output:** A modifiée, P

```
1  $\alpha = (1 + \sqrt{17})/8$ 
2  $k = 1$ 
3 while  $k \leq n$  do
4   @viewA[k : n, k : n]
5    $\mu_1 = \max_i |a_{ii}| := |a_{rr}|$ 
6    $\mu_0 = \max_{ij} |a_{ij}| := |a_{pq}|$ 
7   if  $\mu \geq \alpha\mu_0$  then
8      $a_{rr}$  est le pivot  $1 \times 1$ ,  $s = 1$ 
9     Permuter les lignes et colonnes  $k$  et  $r$  dans  $A$ 
10    Permuter les entrées  $k$  et  $r$  dans  $P$ 
11    Insérer le bloc  $E$  dans  $A[k, k]$ 
12    Insérer la colonne  $L$  dans  $A[(k + 1) : n, k]$ 
13    Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
14     $k += 1$ 
15    Mettre à jour la vue sur  $A$ 
16  else
17     $a_{pq}$  est le pivot  $2 \times 2$ ,  $s = 2$ 
18    Permuter les lignes et colonnes  $k$  et  $p$  dans  $A$ 
19    Permuter les lignes et colonnes  $k + 1$  et  $q$  dans  $A$ 
20    Permuter les entrées  $k$  et  $p$  dans  $P$ 
21    Permuter les entrées  $k + 1$  et  $q$  dans  $P$ 
22    Insérer le bloc  $E$  dans  $A[k : (k + 1), k : (k + 1)]$ 
23    Insérer les colonnes de  $L$  dans  $A[(k + 1) : n, k : (k + 1)]$ 
24    Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
25     $k += 2$ 
26    Mettre à jour la vue sur  $A$ 
27  end
28 end
```

2.3 Bunch-Kaufman

La stratégie de Bunch-Kaufman pour le choix du pivot est aussi appelée *pivotage partiel*. Comme dans la stratégie de Bunch-Parlett, elle se base sur les valeurs maximales des entrées de la matrice sur et hors diagonale. La différence repose dans le fait qu'elle évite le scannage complet de la matrice pour faire son choix, le décortiquant plutôt en 3 sections de recherche beaucoup plus petite. Le choix du pivot est effectué comme suit;

Poser $\alpha = (1 + \sqrt{17})/8$

Poser $\omega_1 = \max_i |a_{i1}|$ pour i sous diagonale

Si $\omega_1 = 0$, passer à la sous-matrice suivante.

Sinon

Si $|a_{11}| \geq \alpha\omega_1$

Utiliser a_{11} comme pivot 1×1 ($s = 1$, aucune permutation)

Sinon

Poser r , l'index de la valeur maximale dans la colonne 1

Poser $\omega_r = \max_i |a_{ir}|$ pour i sous diagonale

Si $|a_{11}|\omega_r \geq \alpha\omega_1^2$

Utiliser a_{11} comme pivot 1×1 ($s = 1$, aucune permutation)

Sinon

Si $|a_{rr}| \geq \alpha\omega_r$

Utiliser a_{rr} comme pivot 1×1 ($s = 1$, les colonnes et rangées 1 et r sont permutées)

Sinon

Utiliser $\begin{bmatrix} a_{11} & a_{r1} \\ a_{r1} & a_{rr} \end{bmatrix}$ comme pivot 2×2 ($s = 2$, les colonnes et rangées 2 et r sont permutées)

On peut visualiser l'emplacement des pivots comme suit;

$$\begin{pmatrix} a_{11} & \cdots & a_{r1}(\omega_1) & \cdots & \cdots & \cdots \\ \vdots & & \vdots & & & \\ a_{r1}(\omega_1) & \cdots & a_{rr} & \cdots & a_{ir}(\omega_r) & \cdots \\ \vdots & & \vdots & & & \\ \vdots & & a_{ir}(\omega_r) & & & \\ \vdots & & \vdots & & & \end{pmatrix}$$

2.3.1 Pseudo-code

Pour l'implémentation, on reprend la majorité de ce qui a été fait pour Bunch-Parlett, mais en ajustant le choix de p . Le pseudo-code suivant illustre la stratégie d'implémentation *in-place* de cet algorithme, présentée dans la Section 3.

Algorithm 2: Factorisation de Bunch-Kaufman (pivotage partiel)

Result: $PAP^T = LBL^T$ **Input :** A hermitienne indéfinie**Output:** A modifiée, P

```
1  $\alpha = (1 + \sqrt{17})/8$ 
2  $k = 1$ 
3 while  $k < n - 1$  do
4   @viewA[ $k : n, k : n$ ]
5    $\omega_1 = \max_i |a_{i1}|$  pour  $i$  sous la diagonale
6   if  $\omega_1 = 0$  then
7     Passer à la sous-matrice suivante
8   else
9     if  $|a_{11}| \geq \alpha\omega_1$  then
10       $a_{11}$  est le pivot  $1 \times 1$ ,  $s = 1$ 
11      Insérer le bloc  $E$  dans  $A[k, k]$ 
12      Insérer la colonne  $L$  dans  $A[(k + 1) : n, k]$ 
13      Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
14       $k += 1$ 
15      Mettre à jour la vue sur  $A$ 
16    else
17       $r = \text{index}(\max_i |a_{i1}|)$ 
18       $\omega_r = \max_i |a_{ir}|$  pour  $i$  sous la diagonale
19      if  $|a_{11}|\omega_r \geq \alpha\omega_1^2$  then
20         $a_{11}$  est le pivot  $1 \times 1$ ,  $s = 1$ 
21        Insérer le bloc  $E$  dans  $A[k, k]$ 
22        Insérer la colonne  $L$  dans  $A[(k + 1) : n, k]$ 
23        Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
24         $k += 1$ 
25        Mettre à jour la vue sur  $A$ 
26      else
27        if  $|a_{rr}| \geq \alpha\omega_r$  then
28           $a_{rr}$  est le pivot  $1 \times 1$ ,  $s = 1$ 
29          Permuter les lignes et colonnes  $k$  et  $r$  dans  $A$ 
30          Permuter les entrées  $k$  et  $r$  dans  $P$ 
31          Insérer le bloc  $E$  dans  $A[k, k]$ 
32          Insérer la colonne  $L$  dans  $A[(k + 1) : n, k]$ 
33          Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
34           $k += 1$ 
35          Mettre à jour la vue sur  $A$ 
36        else
37           $a_{r1}$  est le pivot  $2 \times 2$ ,  $s = 2$ 
38          Permuter les lignes et colonnes  $k + 1$  et  $r$  dans  $A$ 
39          Permuter les entrées  $k + 1$  et  $r$  dans  $P$ 
40          Insérer le bloc  $E$  dans  $A[k : (k + 1), k : (k + 1)]$ 
41          Insérer les colonnes de  $L$  dans  $A[(k + 1) : n, k : (k + 1)]$ 
42          Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
43           $k += 2$ 
44          Mettre à jour la vue sur  $A$ 
45        end
46      end
47    end
48  end
49 end
```

2.4 Rook pivoting

La stratégie de *Rook pivoting* est utilisée lorsque l'on veut éviter que la norme de L soit trop grande (ce qui peut générer des instabilités dans certains algorithmes.) Plusieurs éléments du processus sont les mêmes que pour Bunch-Kaufman, mais le processus comporte une portion itérative. Le choix du pivot est effectué comme suit;

Poser $\alpha = (1 + \sqrt{17})/8$

Poser $\omega_1 = \max_i |a_{i1}|$ pour i sous diagonale

Si $\omega_1 = 0$, passer à la sous-matrice suivante.

Sinon

Si $|a_{11}| \geq \alpha\omega_1$

Utiliser a_{11} comme pivot 1×1 ($s = 1$, aucune permutation)

Sinon

$i = 1$

Répéter jusqu'au choix d'un pivot

Poser r , l'index de la valeur maximale dans la colonne i

Poser $\omega_r = \max_i |a_{ir}|$ pour i sous diagonale

Si $|a_{rr}| \geq \alpha\omega_r$

Utiliser a_{rr} comme pivot 1×1 ($s = 1$, les colonnes et rangées 1 et r sont permutées)

Sinon

Si $\omega_i = \omega_r$

Utiliser $\begin{bmatrix} a_{ii} & a_{ri} \\ a_{ri} & a_{rr} \end{bmatrix}$ comme pivot 2×2 ($s = 2$, les colonnes et rangées 1 et i sont permutées, suivies des colonnes et rangées 2 et r)

Sinon

$i = r$, $\omega_i = \omega_r$

Si un pivot 1×1 est trouvé, le pivot est choisi directement. Sinon, le processus cherche un pivot a_{ri} tel que $|a_{ri}|$ est le maximum dans la rangée r et le maximum dans la colonne i .

2.4.1 Pseudo-code

Le pseudo-code est encore une fois semblable à celui des processus précédent, à la différence de la boucle `while` utilisée dans le choix du pivot. Il illustre la stratégie d'implémentation *in-place* de cet algorithme, présentée dans la Section 3.

Algorithm 3: Factorisation avec rook-pivoting

Result: $PAP^T = LBL^T$ **Input :** A hermitienne indéfinie**Output:** A modifiée, P

```
1  $\alpha = (1 + \sqrt{17})/8$ 
2  $k = 1$ 
3 while  $k < n - 1$  do
4    $@view A[k : n, k : n]$ 
5    $\omega_1 = \max_i |a_{i1}|$  pour  $i$  sous la diagonale
6   if  $\omega_1 = 0$  then
7     Passer à la sous-matrice suivante
8   else
9     if  $|a_{11}| \geq \alpha \omega_1$  then
10       $a_{11}$  est le pivot  $1 \times 1$ ,  $s = 1$ 
11      Insérer le bloc  $E$  dans  $A[k, k]$ 
12      Insérer la colonne  $L$  dans  $A[(k + 1) : n, k]$ 
13      Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
14       $k += 1$ 
15      Mettre à jour la vue sur  $A$ 
16    else
17       $j = 1$ 
18      while Un pivot n'a pas été choisi do
19         $r = \text{index}(\max_i |a_{ij}|)$ 
20         $\omega_r = \max_i |a_{ir}|$  pour  $i$  sous la diagonale
21        if  $|a_{rr}| \geq \alpha \omega_r$  then
22           $a_{rr}$  est le pivot  $1 \times 1$ ,  $s = 1$ 
23          Permuter les lignes et colonnes  $k$  et  $r$  dans  $A$ 
24          Permuter les entrées  $k$  et  $r$  dans  $P$ 
25          Insérer le bloc  $E$  dans  $A[k, k]$ 
26          Insérer la colonne  $L$  dans  $A[(k + 1) : n, k]$ 
27          Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
28           $k += 1$ 
29          Mettre à jour la vue sur  $A$ 
30        else
31          if  $\omega_i = \omega_r$  then
32             $a_{rj}$  est le pivot  $2 \times 2$ ,  $s = 2$ 
33            Permuter les lignes et colonnes 1 et  $j$  dans  $A$ 
34            Permuter les lignes et colonnes 2 et  $r$  dans  $A$ 
35            Permuter les entrées 1 et  $j$  dans  $P$ 
36            Permuter les entrées 2 et  $r$  dans  $P$ 
37            Insérer le bloc  $E$  dans  $A[k : (k + 1), k : (k + 1)]$ 
38            Insérer les colonnes de  $L$  dans  $A[(k + 1) : n, k : (k + 1)]$ 
39            Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
40             $k += 2$ 
41            Mettre à jour la vue sur  $A$ 
42          else
43             $j = r$ 
44             $\omega_j = \omega_r$ 
45          end
46        end
47      end
48    end
49  end
50 end
```

3 Implémentation

3.1 Environnement

```
using Pkg
Pkg.activate("Projet_env")
Pkg.add("BenchmarkTools")
Pkg.add("SuiteSparseMatrixCollection")
Pkg.add("MatrixMarket")

using LinearAlgebra, SparseArrays
using MatrixMarket
using SuiteSparseMatrixCollection
using Test
using BenchmarkTools
```

3.2 Stratégies d'optimisation

Différentes stratégies ont été adoptées pour optimiser les performances des fonctions créées, autant d'un point de vue mémoire que vitesse d'exécution.

Tout d'abord, pour se conformer aux normes présentes dans Julia, deux versions de chaque processus ont été implémentées. La première renvoie les matrices L et B ainsi qu'un vecteur des permutations effectuées vec_P au sein d'une struct. La deuxième écrase la matrice A afin de stocker les membres de la factorisation - le nom de cette dernière contient ainsi un ! pour signifier la modification des éléments d'entrée.

```
struct LBL{T}
    """
    Structure pour une matrice factorisée selon PAP' = LBL' (Bunch-Parlett et Bunch-Kaufman)
    """
    L::LowerTriangular{T, Matrix{T}}
    B::Hermitian{T, Tridiagonal{T, Vector{T}}}
    vec_P::Vector{Int}
end
```

La première version permet un accès élégant à une structure contenant toutes les quantités relatives à la factorisation, alors que la seconde version permet de limiter davantage les allocations en stockant directement les matrices B et L dans les emplacements mémoire de A tout au long de celle-ci. Le schéma suivant illustre cela : les blocs bleus représentent les blocs de la matrice B et les verts les colonnes de la matrices L . Toutes les valeurs au dessus de la diagonale sont automatiquement nulles puisque l'on utilise des matrices triangulaires inférieures afin de représenter les matrices hermitiennes traitées.

$$A = \begin{pmatrix} \square & \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square & \square \end{pmatrix} \rightarrow \begin{pmatrix} \square & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \square & \square & 0 & 0 & 0 & 0 & 0 & 0 \\ \square & 0 & \square & 0 & 0 & 0 & 0 & 0 \\ \square & \square & \square & \square & 0 & 0 & 0 & 0 \\ \square & \square & \square & \square & \square & 0 & 0 & 0 \\ \square & \square & \square & \square & \square & \square & 0 & 0 \\ \square & \square & \square & \square & \square & 0 & \square & 0 \\ \square & \square & \square & \square & \square & \square & \square & \square \end{pmatrix}$$

Ensuite, pour être en mesure d'extraire les matrices B et L de cette matrice A finale, la position des blocs de taille 2×2 est stockée dans un vecteur. Une fonction a été créée pour permettre à l'utilisateur d'extraire B et L à partir de ce vecteur et de la matrice A , `extract_L_and_B_from_LBL_inplace`.

```

function extract_L_and_B_from_LBL_inplace(A, vec_2by2)
    """
    Récupère la matrice L et B à partir de la forme compacte de la factorisation LBL'
    Entrée : - Matrice A sous la forme compacte de la factorisation LBL'
              - Vecteur de position des blocs diagonaux 2x2 vec_2by2
    Sortie : - Matrice L triangulaire inférieure
              - Matrice B tridiagonale hermitienne
    """
    n = size(A, 1)
    T = eltype(A)
    subdiagonal = zeros(T, n-1)
    diagonal = zeros(T, n)
    L = LowerTriangular{Matrix{T}}(I, n, n)

    i_diagonal = 1
    while i_diagonal < n - 1
        if i_diagonal in vec_2by2
            # Matrice B
            subdiagonal[i_diagonal], subdiagonal[i_diagonal+1] = A[i_diagonal+1, i_diagonal], 0
            diagonal[i_diagonal], diagonal[i_diagonal + 1] = A[i_diagonal, i_diagonal],
            ↪ A[i_diagonal+1, i_diagonal+1]

            # Matrice L
            L[i_diagonal + 2:n, i_diagonal] .= A[i_diagonal + 2:n, i_diagonal]
            L[i_diagonal + 2:n, i_diagonal + 1] .= A[i_diagonal + 2:n, i_diagonal + 1]

            # Prochain bloc
            i_diagonal += 2
        else
            # Matrice B
            subdiagonal[i_diagonal] = 0
            diagonal[i_diagonal] = A[i_diagonal, i_diagonal]

            # Matrice L
            L[i_diagonal + 1:n, i_diagonal] .= A[i_diagonal + 1:n, i_diagonal]

            # Prochain bloc
            i_diagonal += 1
        end
    end

    # Dernier bloc
    if i_diagonal == n - 1
        subdiagonal[i_diagonal] = A[i_diagonal+1, i_diagonal]
        diagonal[i_diagonal], diagonal[i_diagonal+1] = A[i_diagonal, i_diagonal], A[i_diagonal+1,
        ↪ i_diagonal+1]
    else
        diagonal[i_diagonal] = A[i_diagonal, i_diagonal]
    end

    B = Hermitian(Tridiagonal(subdiagonal, diagonal, conj(subdiagonal)))

    return L, B
end

```

Dans les deux versions, le caractère hermitien de la matrice A en entrée est exploité pour diminuer le stockage mémoire ainsi que le temps d'exécution. En effet, comme mentionné précédemment, la matrice d'entrée est établie comme triangulaire inférieure, ce qui permet de limiter l'espace mémoire. Ce format est compatible avec les matrices hermitiennes, car il suffit de considérer que les éléments du triangle supérieur sont les transposés conjugués des éléments du triangle inférieur. Bien qu'il existe des formats `Symmetric` ou `Hermitian` en Julia, ceux-ci présentent des blocages internes contre les modifications de leurs éléments, ce qui n'est pas pratique pour le traitement optimal de la factorisation. Le type `LowerTriangular` n'a pas de tels blocages, d'où son utilisation ici. Qui plus est, la matrice L est aussi établie comme triangulaire inférieure de par sa nature, et la matrice B est établie comme hermitienne tridiagonale afin de minimiser l'espace mémoire en ne stockant que la diagonale et la sous-diagonale. Pour B , le type `Hermitian` peut être utilisé puisque l'on a jamais besoin de modifier ses éléments directement. Par ailleurs, les permutations effectuées sont stockées dans un vecteur vec_P plutôt que dans une matrice de permutation complète P , ce qui réduit également l'espace mémoire utilisé. Des fonctions personnalisées ont été créées afin de pouvoir appliquer l'action de P et de P^T sur une matrice donnée seulement à partir de vec_P , de transposer ce dernier et d'en extraire la matrice P correspondante si besoin.

En outre, on note que les algorithmes de factorisation en eux-mêmes ont été conçus et optimisés afin d'exploiter le caractère hermitien des matrices auxquelles ils s'appliquent. Par exemple, lors de la recherche des valeurs absolues maximales hors diagonales pour Bunch-Parlett, seule la moitié inférieure de la matrice a besoin d'être parcourue. On ne regarde donc que ce qui est sous la diagonale, car ce qui est au-dessus n'en est que le transposé conjugué.

Sinon, des `views` sont utilisées pour limiter le nombre d'allocations et des boucles `for` sont fréquemment utilisées pour faire des modifications éléments par éléments et ainsi éviter la création de vecteurs temporaires. Par exemple, pour les pivots de taille 2×2 , le complément de Schur est calculé via son expression algébrique pour éviter les allocations de vecteurs. Il en va de même pour la mise à jour de L dans le cas 2×2 . De plus, des variables temporaires sont souvent utilisées afin d'éviter de recalculer des quantités.

Finalement, il est à noter que la notation `i_diagonal` a été préférée à k dans le code pour éviter toute confusion avec une troisième dimension ou avec un indice de boucle. Le plein potentiel de la notation Latex (α , μ , etc.) n'a pas pu être exploité au sein du code, car ces symboles n'apparaissent pas lors de la production des rapports Quarto.

3.3 Bunch-Parlett

Les fonctions suivantes implémentent la factorisation de Bunch-Parlett (pivotage complet) selon les deux approches mentionnées précédemment.

```
function bunch_parlett(A::LowerTriangular)
    """
    Factorise A selon la factorisation LBL' de Bunch-Parlett, qui utilise la technique de pivotage
    complet.
    Entrée : - Matrice A carrée, hermitienne et indéfinie
    Sortie : - Structure qui contient :
                - Matrice L triangulaire inférieure
                - Matrice B tridiagonale hermitienne
                - Vecteur de permutation vec_P
    """
    n = size(A, 1)
    T = eltype(A)
    subdiagonal = zeros(T, n-1)
    diagonal = zeros(T, n)
    L = LowerTriangular{Matrix{T}}(I, n, n)
    vec_P = collect(1:n)
```

```

 $\alpha = (\text{one}(T) + \text{sqrt}(T(17))) / T(8)$ 
i_diagonal = 1
while i_diagonal < n - 1
    ### Initialisation de la partie de A traitée
    A_ = view(A, i_diagonal:n, i_diagonal:n)
    n_ = size(A_, 1)

    ### Choix du pivot et pivotage
     $\mu_1 = -\text{one}(T)$ 
    r = 1
    for i in 1:n_
        magnitude_ail = abs(A_[i, i])
        if magnitude_ail >  $\mu_1$ 
             $\mu_1 = \text{magnitude\_ail}$ 
            r = i
        end
    end

     $\mu_0 = -\text{one}(T)$ 
    p, q = 1, 1
    for i in 1:n_
        for j in i+1:n_
            magnitude_ij = abs(A_[i, j])
            if magnitude_ij >  $\mu_0$ 
                 $\mu_0 = \text{magnitude\_ij}$ 
                p, q = i, j
            end
        end
    end

    if  $\mu_1 \geq \alpha * \mu_0$  # E de taille 1, P permute 1 et r
        # Permutation dans A_
        perm_r1_et_r2!(A_, 1, r)

        # Permutation dans L
        for j in 1:i_diagonal-1
            L[i_diagonal, j], L[i_diagonal+r-1, j] = L[i_diagonal+r-1, j], L[i_diagonal, j]
        end

        # Permutation dans P
        vec_P[i_diagonal], vec_P[i_diagonal+r-1] = vec_P[i_diagonal+r-1], vec_P[i_diagonal]

        # Déterminant de E
        a_11_conj = conj(A_[1, 1])
        inv_det_E = a_11_conj/ $\mu_1^2$ 

        # Complément de Schur
        for i in 2:n_
            a_i1 = A_[i, 1]
            for j in 2:i
                a_j1_conj = conj(A_[j, 1])
                A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
            end
        end
    end
end

```

```

# Calcul de B
diagonal[i_diagonal] = A_[1, 1]
subdiagonal[i_diagonal] = 0

# Calcul de L
for i in 2:n_
    L[i_diagonal+i-1, i_diagonal] = inv_det_E*A_[i, 1]
end

# Prochain bloc
i_diagonal += 1
else # E de taille 2, P permute 1 et p, puis 2 et q
# Permutation dans A_
perm_r1_et_r2!(A_, 1, p)
perm_r1_et_r2!(A_, 2, q)

# Permutation dans L
for j in 1:i_diagonal-1
    L[i_diagonal, j], L[i_diagonal+p-1, j] = L[i_diagonal+p-1, j], L[i_diagonal, j]
end
for j in 1:i_diagonal-1
    L[i_diagonal+1, j], L[i_diagonal+q-1, j] = L[i_diagonal+q-1, j], L[i_diagonal+1, j]
end

# Permutation dans P
vec_P[i_diagonal], vec_P[i_diagonal+p-1] = vec_P[i_diagonal+p-1], vec_P[i_diagonal]
vec_P[i_diagonal+1], vec_P[i_diagonal+q-1] = vec_P[i_diagonal+q-1], vec_P[i_diagonal+1]

# Déterminant de E
e_11, e_22, e_21 = A_[1, 1], A_[2, 2], A_[2, 1]
e_12 = conj(e_21)
det_E = (e_11*e_22 - e_21*e_12)
det_E_conj = conj(det_E)
magnitude_det_E = abs(det_E)
inv_det_E = det_E_conj/magnitude_det_E^2

# Complément de Schur
for i in 3:n_
    a_i1, a_i2 = A_[i, 1], A_[i, 2]
    for j in 3:i
        a_j1_conj, a_j2_conj = conj(A_[j, 1]), conj(A_[j, 2])
        A_[i, j] -= inv_det_E*((a_i1*e_22 - a_i2*e_21)*a_j1_conj + (a_i2*e_11 -
↪ a_i1*e_12)*a_j2_conj)
    end
end

# Calcul de B
subdiagonal[i_diagonal], subdiagonal[i_diagonal+1] = A_[2, 1], 0
diagonal[i_diagonal], diagonal[i_diagonal + 1] = A_[1, 1], A_[2, 2]

# Calcul de L
for i in i_diagonal + 2:n
    a_i1, a_i2 = A[i, i_diagonal], A[i, i_diagonal+1]

```

```

        L[i, i_diagonal] = inv_det_E*(a_i1*e_22 - a_i2*e_21)
        L[i, i_diagonal+1] = inv_det_E*(a_i2*e_11 - a_i1*e_12)
    end

    # Prochain bloc
    i_diagonal += 2
end

end

if i_diagonal == n - 1
    subdiagonal[i_diagonal] = A[i_diagonal+1, i_diagonal]
    diagonal[i_diagonal], diagonal[i_diagonal+1] = A[i_diagonal, i_diagonal], A[i_diagonal+1,
↪ i_diagonal+1]
else
    diagonal[i_diagonal] = A[i_diagonal, i_diagonal]
end

B = Hermitian(Tridiagonal(subdiagonal, diagonal, conj(subdiagonal)))
result = LBL(L, B, vec_P)

return result
end

function bunch_parlett!(A::LowerTriangular)
    """
    Modifie A sous la forme compacte de la factorisation LBL' de Bunch-Parlett, qui utilise la
    ↪ technique du pivotage complet.
    L et B sont stockés en écrasant A, où L se situe en dessous des éléments blocs diagonaux de B.
    Entrée : - Matrice A carrée, hermitienne et indéfinie
    Sortie : - Vecteur de permutation vec_P
              - Vecteur de position des blocs diagonaux 2x2 vec_2by2
    """
    n = size(A, 1)
    T = eltype(A)
    vec_P = collect(1:n)
    vec_2by2 = Int[]

    α = (one(T) + sqrt(T(17))) / T(8)
    i_diagonal = 1
    while i_diagonal < n - 1
        ### Initialisation de la partie de A traitée
        A_ = view(A, i_diagonal:n, i_diagonal:n)
        n_ = size(A_, 1)

        ### Choix du pivot et pivotage
        μ₁ = -one(T)
        r = 1
        for i in 1:n_
            magnitude_a_ii = abs(A_[i, i])
            if magnitude_a_ii > μ₁
                μ₁ = magnitude_a_ii
                r = i
            end
        end
        i_diagonal += 2
    end
end

```

```

 $\mu_0 = -one(T)$ 
p, q = 1, 1
for i in 1:n_
    for j in i+1:n_
        magnitude_ajj = abs(A_[i, j])
        if magnitude_ajj >  $\mu_0$ 
             $\mu_0 = \text{magnitude\_ajj}$ 
            p, q = i, j
        end
    end
end

if  $\mu_1 \geq \alpha * \mu_0$  # E de taille 1, P permute 1 et r
    # Permutation dans A_
    perm_r1_et_r2!(A_, 1, r)

    # Permutation dans L
    for j in 1:i_diagonal-1
        A[i_diagonal, j], A[i_diagonal+r-1, j] = A[i_diagonal+r-1, j], A[i_diagonal, j]
    end

    # Permutation dans P
    vec_P[i_diagonal], vec_P[i_diagonal+r-1] = vec_P[i_diagonal+r-1], vec_P[i_diagonal]

    # Déterminant de E
    a_11_conj = conj(A_[1, 1])
    inv_det_E = a_11_conj/ $\mu_1^2$ 

    # Complément de Schur
    for i in 2:n_
        a_i1 = A_[i, 1]
        for j in 2:i
            a_j1_conj = conj(A_[j, 1])
            A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
        end
    end

    # Calcul de L
    for i in 2:n_
        A_[i, 1] *= inv_det_E
    end

    # Prochain bloc
    i_diagonal += 1
else # E de taille 2, P permute 1 et p, puis 2 et q
    push!(vec_2by2, i_diagonal)

    # Permutation dans A_
    perm_r1_et_r2!(A_, 1, p)
    perm_r1_et_r2!(A_, 2, q)

    # Permutation dans L
    for j in 1:i_diagonal-1

```



```

        A[i_diagonal, j], A[i_diagonal+p-1, j] = A[i_diagonal+p-1, j], A[i_diagonal, j]
    end
    for j in 1:i_diagonal-1
        A[i_diagonal+1, j], A[i_diagonal+q-1, j] = A[i_diagonal+q-1, j], A[i_diagonal+1, j]
    end

    # Permutation dans P
    vec_P[i_diagonal], vec_P[i_diagonal+p-1] = vec_P[i_diagonal+p-1], vec_P[i_diagonal]
    vec_P[i_diagonal+1], vec_P[i_diagonal+q-1] = vec_P[i_diagonal+q-1], vec_P[i_diagonal+1]

    # Déterminant de E
    e_11, e_22, e_21 = A_[1, 1], A_[2, 2], A_[2, 1]
    e_12 = conj(e_21)
    det_E = (e_11*e_22 - e_21*e_12)
    det_E_conj = conj(det_E)
    magnitude_det_E = abs(det_E)
    inv_det_E = det_E_conj/magnitude_det_E^2

    # Complément de Schur
    for i in 3:n_
        a_i1, a_i2 = A_[i, 1], A_[i, 2]
        for j in 3:i
            a_j1_conj, a_j2_conj = conj(A_[j, 1]), conj(A_[j, 2])
            A_[i, j] -= inv_det_E*((a_i1*e_22 - a_i2*e_21)*a_j1_conj + (a_i2*e_11 -
↪ a_i1*e_12)*a_j2_conj)
        end
    end

    # Calcul de L
    for i in 3:n_
        a_i1, a_i2 = A_[i, 1], A_[i, 2]
        A_[i, 1] = inv_det_E*(a_i1*e_22 - a_i2*e_21)
        A_[i, 2] = inv_det_E*(a_i2*e_11 - a_i1*e_12)
    end

    # Prochain bloc
    i_diagonal += 2
end
end

return vec_P, vec_2by2
end

```

3.4 Bunch-Kaufman

Les fonctions suivantes implémentent la factorisation de Bunch-Kaufman (pivotage partiel) selon les deux approches mentionnées précédemment.

```

function bunch_kaufman(A::LowerTriangular)
    """
    Factorise A selon la factorisation LBL' de Bunch-Kaufman, qui utilise la technique de pivotage
    ↪ partiel
    Entrée : - Matrice A carrée, hermitienne et indéfinie
    Sortie : - Structure qui contient :
    """
end

```

```

- Matrice L triangulaire inférieure
- Matrice B tridiagonale hermitienne
- Vecteur de permutation vec_P
"""
n = size(A, 1)
T = eltype(A)
subdiagonal = zeros(T, n-1)
diagonal = zeros(T, n)
L = LowerTriangular{Matrix{T}}(I, n, n)
vec_P = collect(1:n)

 $\alpha$  = (one(T) + sqrt(T(17))) / T(8)
i_diagonal = 1
while i_diagonal < n - 1
    ### Initialisation de la partie de A traitée
    A_ = view(A, i_diagonal:n, i_diagonal:n)
    n_ = size(A_, 1)

    ### Choix du pivot et pivotage
     $\omega_1$  = -one(T)
    r = 1
    for i in 2:n_
        magnitude_a_i1 = abs(A_[i, 1])
        if magnitude_a_i1 >  $\omega_1$ 
             $\omega_1$  = magnitude_a_i1
            r = i
        end
    end

    magnitude_a_11 = abs(A_[1, 1])
    if magnitude_a_11 >=  $\alpha * \omega_1$  # E de taille 1, P ne permute rien
        # Déterminant de E
        a_11_conj = conj(A_[1, 1])
        inv_det_E = a_11_conj/magnitude_a_11^2

        # Complément de Schur
        for i in 2:n_
            a_i1 = A_[i, 1]
            for j in 2:i
                a_j1_conj = conj(A_[j, 1])
                A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
            end
        end

        # Calcul de B
        diagonal[i_diagonal] = A_[1, 1]
        subdiagonal[i_diagonal] = 0

        # Calcul de L
        for i in 2:n_
            L[i_diagonal+i-1, i_diagonal] = inv_det_E*A_[i, 1]
        end

        # Prochain bloc

```

```

        i_diagonal += 1
    else
         $\omega_r$  = -one(T)
        for j in 1:r-1
            magnitude_a_ir = abs(A_[r, j])
            if magnitude_a_ir >  $\omega_r$ 
                 $\omega_r$  = magnitude_a_ir
            end
        end
        for i in r+1:n_
            magnitude_a_ir = abs(A_[i, r])
            if magnitude_a_ir >  $\omega_r$ 
                 $\omega_r$  = magnitude_a_ir
            end
        end
    end

    if magnitude_a_11* $\omega_r$  >=  $\alpha*\omega_1^2$  # E de taille 1, P ne permute rien
        # Déterminant de E
        a_11_conj = conj(A_[1, 1])
        inv_det_E = a_11_conj/magnitude_a_11^2

        # Complément de Schur
        for i in 2:n_
            a_i1 = A_[i, 1]
            for j in 2:i
                a_j1_conj = conj(A_[j, 1])
                A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
            end
        end

        # Calcul de B
        diagonal[i_diagonal] = A_[1, 1]
        subdiagonal[i_diagonal] = 0

        # Calcul de L
        for i in 2:n_
            L[i_diagonal+i-1, i_diagonal] = inv_det_E*A_[i, 1]
        end

        # Prochain bloc
        i_diagonal += 1
    else
        magnitude_a_rr = abs(A_[r, r])
        if magnitude_a_rr >=  $\alpha*\omega_r$  # E de taille 1, P permute 1 et r
            # Permutation dans A_
            perm_r1_et_r2!(A_, 1, r)

            # Permutation dans L
            for j in 1:i_diagonal-1
                L[i_diagonal, j], L[i_diagonal+r-1, j] = L[i_diagonal+r-1, j], L[i_diagonal, j]
            end

            # Permutation dans P
            vec_P[i_diagonal], vec_P[i_diagonal+r-1] = vec_P[i_diagonal+r-1],
↪ vec_P[i_diagonal]

```

```

# Déterminant de E
a_11_conj = conj(A_[1, 1])
inv_det_E = a_11_conj/magnitude_a_rr^2

# Complément de Schur
for i in 2:n_
    a_i1 = A_[i, 1]
    for j in 2:i
        a_j1_conj = conj(A_[j, 1])
        A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
    end
end

# Calcul de B
diagonal[i_diagonal] = A_[1, 1]
subdiagonal[i_diagonal] = 0

# Calcul de L
for i in 2:n_
    L[i_diagonal+i-1, i_diagonal] = inv_det_E*A_[i, 1]
end

# Prochain bloc
i_diagonal += 1
else # E de taille 2, P permute 2 et r
    # Permutation dans A_
    perm_r1_et_r2!(A_, 2, r)

    # Permutation dans L
    for j in 1:i_diagonal-1
        L[i_diagonal+1, j], L[i_diagonal+r-1, j] = L[i_diagonal+r-1, j],
↪ L[i_diagonal+1, j]
    end

    # Permutation dans P
    vec_P[i_diagonal+1], vec_P[i_diagonal+r-1] = vec_P[i_diagonal+r-1],
↪ vec_P[i_diagonal+1]

# Déterminant de E
e_11, e_22, e_21 = A_[1, 1], A_[2, 2], A_[2, 1]
e_12 = conj(e_21)
det_E = (e_11*e_22 - e_21*e_12)
det_E_conj = conj(det_E)
magnitude_det_E = abs(det_E)
inv_det_E = det_E_conj/magnitude_det_E^2

# Complément de Schur
for i in 3:n_
    a_i1, a_i2 = A_[i, 1], A_[i, 2]
    for j in 3:i
        a_j1_conj, a_j2_conj = conj(A_[j, 1]), conj(A_[j, 2])
        A_[i, j] -= inv_det_E*((a_i1*e_22 - a_i2*e_21)*a_j1_conj + (a_i2*e_11 -
↪ a_i1*e_12)*a_j2_conj)

```

```

        end
    end

    # Calcul de B
    subdiagonal[i_diagonal], subdiagonal[i_diagonal+1] = A_[2, 1], 0
    diagonal[i_diagonal], diagonal[i_diagonal + 1] = A_[1, 1], A_[2, 2]

    # Calcul de L
    for i in i_diagonal + 2:n
        a_i1, a_i2 = A[i, i_diagonal], A[i, i_diagonal+1]
        L[i, i_diagonal] = inv_det_E*(a_i1*e_22 - a_i2*e_21)
        L[i, i_diagonal+1] = inv_det_E*(a_i2*e_11 - a_i1*e_12)
    end

    # Prochain bloc
    i_diagonal += 2
end
end
end

# Dernier bloc
if i_diagonal == n - 1
    subdiagonal[i_diagonal] = A[i_diagonal+1, i_diagonal]
    diagonal[i_diagonal], diagonal[i_diagonal+1] = A[i_diagonal, i_diagonal], A[i_diagonal+1,
↪ i_diagonal+1]
else
    diagonal[i_diagonal] = A[i_diagonal, i_diagonal]
end

B = Hermitian(Tridiagonal(subdiagonal, diagonal, conj(subdiagonal)))
result = LBL(L, B, vec_P)
return result
end

function bunch_kaufman!(A::LowerTriangular)
    """
    Modifie A sous la forme compacte de la factorisation LBL' de Bunch-Kaufman, qui utilise la
    ↪ technique du pivotage partiel.
    L et B sont stockés en écrasant A, où L se situe en dessous des éléments blocs diagonaux de B.
    Entrée : - Matrice A carrée, hermitienne et indéfinie
    Sortie : - Vecteur de permutation vec_P
              - Vecteur de position des blocs diagonaux 2x2 vec_2by2
    """
    n = size(A, 1)
    T = eltype(A)
    vec_P = collect{1:n}
    vec_2by2 = Int[]

    α = (one(T) + sqrt(T(17))) / T(8)
    i_diagonal = 1
    while i_diagonal < n - 1
        ### Initialisation de la partie de A traitée
        A_ = view(A, i_diagonal:n, i_diagonal:n)
    end
end

```

```

n_ = size(A_, 1)

### Choix du pivot et pivotage
 $\omega_1$  = -one(T)
r = 1
for i in 2:n_
    magnitude_a_i1 = abs(A_[i, 1])
    if magnitude_a_i1 >  $\omega_1$ 
         $\omega_1$  = magnitude_a_i1
        r = i
    end
end

magnitude_a_11 = abs(A_[1, 1])
if magnitude_a_11 >=  $\alpha * \omega_1$  # E de taille 1, P ne permute rien
    # Déterminant de E
    a_11_conj = conj(A_[1, 1])
    inv_det_E = a_11_conj/magnitude_a_11^2

    # Complément de Schur
    for i in 2:n_
        a_i1 = A_[i, 1]
        for j in 2:i
            a_j1_conj = conj(A_[j, 1])
            A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
        end
    end

    # Calcul de L
    for i in 2:n_
        A_[i, 1] *= inv_det_E
    end

    # Prochain bloc
    i_diagonal += 1
else
     $\omega_r$  = -one(T)
    for j in 1:r-1
        magnitude_a_ir = abs(A_[r, j])
        if magnitude_a_ir >  $\omega_r$ 
             $\omega_r$  = magnitude_a_ir
        end
    end
    for i in r+1:n_
        magnitude_a_ir = abs(A_[i, r])
        if magnitude_a_ir >  $\omega_r$ 
             $\omega_r$  = magnitude_a_ir
        end
    end

    if magnitude_a_11* $\omega_r$  >=  $\alpha * \omega_1^2$  # E de taille 1, P ne permute rien
        # Déterminant de E
        a_11_conj = conj(A_[1, 1])
        inv_det_E = a_11_conj/magnitude_a_11^2

```

```

# Complément de Schur
for i in 2:n_
    a_i1 = A_[i, 1]
    for j in 2:i
        a_j1_conj = conj(A_[j, 1])
        A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
    end
end

# Calcul de L
for i in 2:n_
    A_[i, 1] *= inv_det_E
end

# Prochain bloc
i_diagonal += 1
else
    magnitude_a_rr = abs(A_[r, r])
    if magnitude_a_rr >=  $\alpha \omega_r$  # E de taille 1, P permute 1 et r
        # Permutation dans A_
        perm_r1_et_r2!(A_, 1, r)

        # Permutation dans L
        for j in 1:i_diagonal-1
            A[i_diagonal, j], A[i_diagonal+r-1, j] = A[i_diagonal+r-1, j], A[i_diagonal, j]
        end

        # Permutation dans P
        vec_P[i_diagonal], vec_P[i_diagonal+r-1] = vec_P[i_diagonal+r-1],
        ↪ vec_P[i_diagonal]

        # Déterminant de E
        a_11_conj = conj(A_[1, 1])
        inv_det_E = a_11_conj/magnitude_a_rr^2

        # Complément de Schur
        for i in 2:n_
            a_i1 = A_[i, 1]
            for j in 2:i
                a_j1_conj = conj(A_[j, 1])
                A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
            end
        end

        # Calcul de L
        for i in 2:n_
            A_[i, 1] *= inv_det_E
        end

        # Prochain bloc
        i_diagonal += 1
    else # E de taille 2, P permute 2 et r
        push!(vec_2by2, i_diagonal)

```

```

        # Permutation dans A_
        perm_r1_et_r2!(A_, 2, r)

        # Permutation dans L
        for j in 1:i_diagonal-1
            A[i_diagonal+1, j], A[i_diagonal+r-1, j] = A[i_diagonal+r-1, j],
↪ A[i_diagonal+1, j]
        end

        # Permutation dans P
        vec_P[i_diagonal+1], vec_P[i_diagonal+r-1] = vec_P[i_diagonal+r-1],
↪ vec_P[i_diagonal+1]

        # Déterminant de E
        e_11, e_22, e_21 = A_[1, 1], A_[2, 2], A_[2, 1]
        e_12 = conj(e_21)
        det_E = (e_11*e_22 - e_21*e_12)
        det_E_conj = conj(det_E)
        magnitude_det_E = abs(det_E)
        inv_det_E = det_E_conj/magnitude_det_E^2

        # Complément de Schur
        for i in 3:n_
            a_i1, a_i2 = A_[i, 1], A_[i, 2]
            for j in 3:i
                a_j1_conj, a_j2_conj = conj(A_[j, 1]), conj(A_[j, 2])
                A_[i, j] -= inv_det_E*((a_i1*e_22 - a_i2*e_21)*a_j1_conj + (a_i2*e_11 -
↪ a_i1*e_12)*a_j2_conj)
            end
        end

        # Calcul de L
        for i in 3:n_
            a_i1, a_i2 = A_[i, 1], A_[i, 2]
            A_[i, 1] = inv_det_E*(a_i1*e_22 - a_i2*e_21)
            A_[i, 2] = inv_det_E*(a_i2*e_11 - a_i1*e_12)
        end

        # Prochain bloc
        i_diagonal += 2
    end
end
end
end

return vec_P, vec_2by2
end

```

3.5 Rook pivoting

Les fonctions suivantes implémentent la factorisation avec *rook pivoting* selon les deux approches mentionnées précédemment.


```

function bunch_kaufman_rook(A::LowerTriangular)
    """
    Factorise A selon la factorisation LBL' de Bunch-Kaufman, qui utilise la technique de pivotage
    ↪ partiel
    Entrée : - Matrice A carrée, hermitienne et indéfinie
    Sortie : - Structure qui contient :
                - Matrice L triangulaire inférieure
                - Matrice B tridiagonale hermitienne
                - Vecteur de permutation vec_P
    """
    n = size(A, 1)
    T = eltype(A)
    subdiagonal = zeros(T, n-1)
    diagonal = zeros(T, n)
    L = LowerTriangular(Matrix{T}(I, n, n))
    vec_P = collect(1:n)

    α = (one(T) + sqrt(T(17))) / T(8)
    i_diagonal = 1
    while i_diagonal < n - 1
        ### Initialisation de la partie de A traitée
        A_ = view(A, i_diagonal:n, i_diagonal:n)
        n_ = size(A_, 1)

        ### Choix du pivot et pivotage
        ω₁ = -one(T)
        r = 1
        for i in 2:n_
            magnitude_a_i1 = abs(A_[i, 1])
            if magnitude_a_i1 > ω₁
                ω₁ = magnitude_a_i1
                r = i
            end
        end

        magnitude_a_11 = abs(A_[1, 1])
        if magnitude_a_11 >= α*ω₁ # E de taille 1, P ne permute rien
            # Déterminant de E
            a_11_conj = conj(A_[1, 1])
            inv_det_E = a_11_conj/magnitude_a_11^2

            # Complément de Schur
            for i in 2:n_
                a_i1 = A_[i, 1]
                for j in 2:i
                    a_j1_conj = conj(A_[j, 1])
                    A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
                end
            end

            # Calcul de B
            diagonal[i_diagonal] = A_[1, 1]
            subdiagonal[i_diagonal] = 0
        end
        i_diagonal += 1
    end
end

```

```

# Calcul de L
for i in 2:n_
    L[i_diagonal+i-1, i_diagonal] = inv_det_E*A_[i, 1]
end

# Prochain bloc
i_diagonal += 1
else
    index = 1
    w_index =  $\omega_1$ 
    r_new = r
    repeat = true
    while repeat
         $\omega_r$  = -one(T)
        # Subdiagonal
        for i in r+1:n_
            magnitude_a_ir = abs(A_[i, r])
            if magnitude_a_ir >  $\omega_r$ 
                 $\omega_r$  = magnitude_a_ir
                r_new = i
            end
        end
        # Superdiagonal
        for j in 1:r-1
            magnitude_a_ir = abs(A_[r, j])
            if magnitude_a_ir >  $\omega_r$ 
                 $\omega_r$  = magnitude_a_ir
            end
        end
        magnitude_a_rr = abs(A_[r, r])
        if magnitude_a_rr >=  $\alpha \omega_r$  # E de taille 1, P permute 1 et r
            repeat = false

            # Permutation dans A_
            perm_r1_et_r2!(A_, 1, r)

            # Permutation dans L
            for j in 1:i_diagonal-1
                L[i_diagonal, j], L[i_diagonal+r-1, j] = L[i_diagonal+r-1, j], L[i_diagonal, j]
            end

            # Permutation dans P
            vec_P[i_diagonal], vec_P[i_diagonal+r-1] = vec_P[i_diagonal+r-1],
↪ vec_P[i_diagonal]

            # Déterminant de E
            a_11_conj = conj(A_[1, 1])
            inv_det_E = a_11_conj/magnitude_a_rr^2

            # Complément de Schur
            for i in 2:n_
                a_i1 = A_[i, 1]
                for j in 2:i

```

```

        a_j1_conj = conj(A_[j, 1])
        A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
    end
end

# Calcul de B
diagonal[i_diagonal] = A_[1, 1]
subdiagonal[i_diagonal] = 0

# Calcul de L
for i in 2:n_
    L[i_diagonal+i-1, i_diagonal] = inv_det_E*A_[i, 1]
end

# Prochain bloc
i_diagonal += 1
elseif w_index == ωr # E de taille 2, P permute 1 et index, puis 2 et r
    repeat = false

    # Permutation dans A_
    perm_r1_et_r2!(A_, 1, index)
    perm_r1_et_r2!(A_, 2, r)

    # Permutation dans L
    for j in 1:i_diagonal-1
        L[i_diagonal, j], L[i_diagonal+index-1, j] = L[i_diagonal+index-1, j],
↪ L[i_diagonal, j]
    end
    for j in 1:i_diagonal-1
        L[i_diagonal+1, j], L[i_diagonal+r-1, j] = L[i_diagonal+r-1, j],
↪ L[i_diagonal+1, j]
    end

    # Permutation dans P
    vec_P[i_diagonal], vec_P[i_diagonal+index-1] = vec_P[i_diagonal+index-1],
↪ vec_P[i_diagonal]
    vec_P[i_diagonal+1], vec_P[i_diagonal+r-1] = vec_P[i_diagonal+r-1],
↪ vec_P[i_diagonal+1]

    # Déterminant de E
    e_11, e_22, e_21 = A_[1, 1], A_[2, 2], A_[2, 1]
    e_12 = conj(e_21)
    det_E = (e_11*e_22 - e_21*e_12)
    det_E_conj = conj(det_E)
    magnitude_det_E = abs(det_E)
    inv_det_E = det_E_conj/magnitude_det_E^2

    # Complément de Schur
    for i in 3:n_
        a_i1, a_i2 = A_[i, 1], A_[i, 2]
        for j in 3:i
            a_j1_conj, a_j2_conj = conj(A_[j, 1]), conj(A_[j, 2])
            A_[i, j] -= inv_det_E*((a_i1*e_22 - a_i2*e_21)*a_j1_conj + (a_i2*e_11 -
↪ a_i1*e_12)*a_j2_conj)

```

```

        end
    end

    # Calcul de B
    subdiagonal[i_diagonal], subdiagonal[i_diagonal+1] = A_[2, 1], 0
    diagonal[i_diagonal], diagonal[i_diagonal + 1] = A_[1, 1], A_[2, 2]

    # Calcul de L
    for i in i_diagonal + 2:n
        a_i1, a_i2 = A[i, i_diagonal], A[i, i_diagonal+1]
        L[i, i_diagonal] = inv_det_E*(a_i1*e_22 - a_i2*e_21)
        L[i, i_diagonal+1] = inv_det_E*(a_i2*e_11 - a_i1*e_12)
    end

    # Prochain bloc
    i_diagonal += 2
else
    index = r
    w_index = w_r
    r = r_new
end
end
end

# Dernier bloc
if i_diagonal == n - 1
    subdiagonal[i_diagonal] = A[i_diagonal+1, i_diagonal]
    diagonal[i_diagonal], diagonal[i_diagonal+1] = A[i_diagonal, i_diagonal], A[i_diagonal+1,
↪ i_diagonal+1]
else
    diagonal[i_diagonal] = A[i_diagonal, i_diagonal]
end

B = Hermitian(Tridiagonal(subdiagonal, diagonal, conj(subdiagonal)))
result = LBL(L, B, vec_P)
return result
end

function bunch_kaufman_rook!(A::LowerTriangular)
    """
    Modifie A sous la forme compacte de la factorisation LBL' de Bunch-Kaufman, qui utilise la
    ↪ technique du 'rook pivoting'.
    L et B sont stockés en écrasant A, où L se situe en dessous des éléments blocs diagonaux de B.
    Entrée : - Matrice A carrée, hermitienne et indéfinie
    Sortie : - Vecteur de permutation vec_P
              - Vecteur de position des blocs diagonaux 2x2 vec_2by2
    """
    n = size(A, 1)
    T = eltype(A)
    vec_P = collect{1:n}
    vec_2by2 = Int[]

    α = (one(T) + sqrt(T(17))) / T(8)

```

```

i_diagonal = 1
while i_diagonal < n - 1
    ### Initialisation de la partie de A traitée
    A_ = view(A, i_diagonal:n, i_diagonal:n)
    n_ = size(A_, 1)

    ### Choix du pivot et pivotage
     $\omega_1$  = -one(T)
    r = 1
    for i in 2:n_
        magnitude_a_i1 = abs(A_[i, 1])
        if magnitude_a_i1 >  $\omega_1$ 
             $\omega_1$  = magnitude_a_i1
            r = i
        end
    end

    magnitude_a_11 = abs(A_[1, 1])
    if magnitude_a_11 >=  $\alpha \omega_1$  # E de taille 1, P ne permute rien
        # Déterminant de E
        a_11_conj = conj(A_[1, 1])
        inv_det_E = a_11_conj/magnitude_a_11^2

        # Complément de Schur
        for i in 2:n_
            a_i1 = A_[i, 1]
            for j in 2:i
                a_j1_conj = conj(A_[j, 1])
                A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
            end
        end

        # Calcul de L
        for i in 2:n_
            A_[i, 1] *= inv_det_E
        end

        # Prochain bloc
        i_diagonal += 1
    else
        index = 1
        w_index =  $\omega_1$ 
        r_new = r
        repeat = true
        while repeat
             $\omega_r$  = -one(T)
            # Subdiagonal
            for i in r+1:n_
                magnitude_a_ir = abs(A_[i, r])
                if magnitude_a_ir >  $\omega_r$ 
                     $\omega_r$  = magnitude_a_ir
                    r_new = i
                end
            end
            repeat = false
        end
    end
end

```

```

# Superdiagonal
for j in 1:r-1
    magnitude_a_ir = abs(A_[r, j])
    if magnitude_a_ir >  $\omega_r$ 
         $\omega_r$  = magnitude_a_ir
    end
end

magnitude_a_rr = abs(A_[r, r])
if magnitude_a_rr >=  $\alpha \omega_r$  # E de taille 1, P permute 1 et r
    repeat = false

    # Permutation dans A_
    perm_r1_et_r2!(A_, 1, r)

    # Permutation dans L
    for j in 1:i_diagonal-1
        A[i_diagonal, j], A[i_diagonal+r-1, j] = A[i_diagonal+r-1, j], A[i_diagonal, j]
    end

    # Permutation dans P
    vec_P[i_diagonal], vec_P[i_diagonal+r-1] = vec_P[i_diagonal+r-1],
    ↪ vec_P[i_diagonal]

    # Déterminant de E
    a_11_conj = conj(A_[1, 1])
    inv_det_E = a_11_conj/magnitude_a_rr^2

    # Complément de Schur
    for i in 2:n_
        a_i1 = A_[i, 1]
        for j in 2:i
            a_j1_conj = conj(A_[j, 1])
            A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
        end
    end

    # Calcul de L
    for i in 2:n_
        A_[i, 1] *= inv_det_E
    end

    # Prochain bloc
    i_diagonal += 1
elseif w_index ==  $\omega_r$  # E de taille 2, P permute 1 et index, puis 2 et r
    repeat = false

    push!(vec_2by2, i_diagonal)

    # Permutation dans A_
    perm_r1_et_r2!(A_, 1, index)
    perm_r1_et_r2!(A_, 2, r)

    # Permutation dans L

```

```

        for j in 1:i_diagonal-1
            A[i_diagonal, j], A[i_diagonal+index-1, j] = A[i_diagonal+index-1, j],
↪ A[i_diagonal, j]
        end
        for j in 1:i_diagonal-1
            A[i_diagonal+1, j], A[i_diagonal+r-1, j] = A[i_diagonal+r-1, j],
↪ A[i_diagonal+1, j]
        end

        # Permutation dans P
        vec_P[i_diagonal], vec_P[i_diagonal+index-1] = vec_P[i_diagonal+index-1],
↪ vec_P[i_diagonal]
        vec_P[i_diagonal+1], vec_P[i_diagonal+r-1] = vec_P[i_diagonal+r-1],
↪ vec_P[i_diagonal+1]

        # Déterminant de E
        e_11, e_22, e_21 = A_[1, 1], A_[2, 2], A_[2, 1]
        e_12 = conj(e_21)
        det_E = (e_11*e_22 - e_21*e_12)
        det_E_conj = conj(det_E)
        magnitude_det_E = abs(det_E)
        inv_det_E = det_E_conj/magnitude_det_E^2

        # Complément de Schur
        for i in 3:n_
            a_i1, a_i2 = A_[i, 1], A_[i, 2]
            for j in 3:i
                a_j1_conj, a_j2_conj = conj(A_[j, 1]), conj(A_[j, 2])
                A_[i, j] -= inv_det_E*((a_i1*e_22 - a_i2*e_21)*a_j1_conj + (a_i2*e_11 -
↪ a_i1*e_12)*a_j2_conj)
            end
        end

        # Calcul de L
        for i in 3:n_
            a_i1, a_i2 = A_[i, 1], A_[i, 2]
            A_[i, 1] = inv_det_E*(a_i1*e_22 - a_i2*e_21)
            A_[i, 2] = inv_det_E*(a_i2*e_11 - a_i1*e_12)
        end

        # Prochain bloc
        i_diagonal += 2
    else
        index = r
        w_index = ωr
        r = r_new
    end
end
end
end
end

return vec_P, vec_2by2
end

```

4 Tests et performances

Trois types de tests ont été effectués sur les implémentations:

1. La validité des factorisations ainsi que leur compatibilité avec différents types de données (`Float64`, `ComplexF64` et `BigFloat`) est établie.
2. Les performances (nombre d'allocations et temps d'exécution) des différentes implémentations sont caractérisées à l'aide du module `BenchmarkTools`.
3. La capacité à résoudre correctement des systèmes linéaires est établie, en particulier sur des systèmes de point de selle.

Dans le fichier `Code`, la section *Utilitaire* liste plusieurs des fonctions nécessaires au roulement de ces tests, en particulier les tests sur les types de données. Les tests sont ensuite présentés dans l'ordre mentionnés, suite aux implémentations.

4.1 Validité et compatibilité

Pour établir la validité des fonctions sur différents types de données, 25 matrices de forme

$$K = \begin{bmatrix} I & A \\ A^* & 0 \end{bmatrix}$$

sont créées l'une après l'autre, où A est aléatoire et de taille 50×25 . Les matrices sont factorisées et leur reconstruction par la factorisation $P^T L B L' P$ est comparé à A via la norme de leur différence. La tolérance des tests est de $1e^{-8}$. Il est aussi validé que le type des données contenues dans L et B est le même que celui des éléments de A . Toutes les implémentations passent les tests en ce qui concerne le type de données supportées.

4.2 Benchmarking

Similairement à précédemment, une matrice K de taille 75×75 est construite à partir d'une matrice A aléatoire. Celle-ci est utilisée en entrée pour effectuer le `Benchmarking` des différentes implémentations. La factorisation est également effectuée en utilisant la fonction `bunchkaufman` de Julia, qui appelle LAPACK. L'option `rook` de cette dernière devait être testée, mais elle ne semble pas être reconnue par la fonction - il semble y avoir un décalage entre la documentation en ligne et le code source de la version de Julia utilisée ici. Les résultats obtenus sont présentés dans la cellule ci-dessous et les résultats commentés dans la Section 5.

Performance de `bunch_parlett`:

- Allocations : 18
- Temps d'exécution : 8.382023607744108e6

Performance de `bunch_parlett!`:

- Allocations : 9
- Temps d'exécution : 6.718659451482479e6

Performance de `bunch_kaufman`:

- Allocations : 18
- Temps d'exécution : 5.398029560260586e6

Performance de `bunch_kaufman!`:

- Allocations : 10
- Temps d'exécution : 5.539060867777778e6

Performance de `bunch_kaufman_rook`:

- Allocations : 18
- Temps d'exécution : 5.813942015151516e6

Performance de `bunch_kaufman_rook!`:

- Allocations : 10

- Temps d'exécution : 5.560475282051282e6

Performance de bunchkaufman LAPACK:

- Allocations : 67
- Temps d'exécution : 4.316763417391304e6

Performance de bunchkaufman! LAPACK:

- Allocations : 61
- Temps d'exécution : 3.6760701345158904e6

4.3 Résolution de systèmes linéaires

Pour la résolution de systèmes linéaires, deux types de tests sont effectués. D'abord, des matrices K hermitiennes indéfinies aléatoires sont construites de la même manière que précédemment, couplées à des vecteurs b qui sont définis de manière aléatoire. Le tout forme des systèmes de point de selle à résoudre, et les solutions obtenues sont comparées aux solutions obtenues par factorisation qr sur les problèmes au moindres carrés en étant la source. La définition de la fonction $\backslash$ a été étendue pour spécifiquement s'appliquer à la structure de la factorisation $PAP^T = LBL'$.

```
import Base: \

function \(F::LBL{T}, b::AbstractVector{T}) where T
    L, B, vec_P = F.L, F.B, F.vec_P
    n = size(vec_P, 1)

    # Permuter b : b_perm = P * b
    b_perm = b[vec_P]

    # Résoudre L * y = b_perm <=> y = L \ b_perm
    y = similar(b)
    for i in 1:n
        y[i] = b_perm[i]
        for j in 1:i-1
            y[i] -= L[i, j]*y[j]
        end
    end

    # Résoudre B * z = y <=> z = B \ y
    z = similar(b)
    i = 1
    while i < n-1
        if B[i+1, i] == 0
            z[i] = y[i]/B[i, i]

            i += 1
        else
            B_11, B_22, B_21 = B[i, i], B[i+1, i+1], B[i+1, i]
            B_12 = conj(B_21)
            det = (B_11*B_22 - B_21*B_12)
            inv_det = conj(det)/abs(det)^2
            z[i] = inv_det*(B_22*y[i] - B_12*y[i+1])
            z[i+1] = inv_det*(B_11*y[i+1] - B_21*y[i])

            i += 2
        end
    end
    if B[n, n-1] == 0
```

```

        z[n-1] = y[n-1]/B[n-1, n-1]
        z[n] = y[n]/B[n, n]
    else
        B_11, B_22, B_21 = B[n-1, n-1], B[n, n], B[n, n-1]
        B_12 = conj(B_21)
        det = (B_11*B_22 - B_21*B_12)
        inv_det = conj(det)/abs(det)^2
        z[n-1] = inv_det*(B_22*y[n-1] - B_12*y[n])
        z[n] = inv_det*(B_11*y[n] - B_21*y[n-1])
    end

    # Résoudre L'* x_perm = z <=> x_perm = L' \ z
    x_perm = similar(b)
    for i in n:-1:1
        x_perm[i] = z[i]
        for j in n:-1:i+1
            x_perm[i] -= conj(L[j, i])*x_perm[j]
        end
    end

    # Permuter x_perm : x = P' * x_perm
    x = similar(b)
    x[vec_P] = x_perm # Inverse de P est P', car P est une permutation

    return x
end

```

Les tests sont effectués sur 100 problèmes aléatoires différents de type `BigFloat` complexe. L'erreur moyenne sur x est présentée ci-dessous.

Average error on solution:

```

- bunch_parlett : 1.965220606144465e-72
- bunch_parlett! : 1.965220606144465e-72
- bunch_kaufman : 1.5246950352785973e-76
- bunch_kaufman! : 1.5246950352785973e-76
- bunch_kaufman_rook : 3.9636552635334506e-74
- bunch_kaufman_rook! : 3.9636552635334506e-74
- bunch_kaufman de LAPACK : 2.7855703676639216e-76
- bunch_kaufman! de LAPACK : 2.7855703676639216e-76

```

Ensuite, quatre problèmes issus de `MatrixMarket` (Suite Sparse Collection) sont résolus avec les différentes implémentations, soient `ch5-5-b1`, `ch4-4-b1`, `ch4-4-b2` et `n3c5-b2`. Les systèmes de point de selle sont générées comme dans les sections précédentes, mais en incluant cette fois-ci une légère perturbation dans le bloc inférieur droit des matrices K lorsqu'il s'agit de résoudre avec les fonctions `bunchkaufman` et `bunchkaufman!` standards de Julia. En effet, autrement, celles-ci lèvent une exception de singularité pour certains problèmes. Ces problèmes ne sont pas tous de rang plein et ainsi la résolution par factorisation ne retourne pas nécessairement la solution de moindre norme. C'est donc la norme du résidu du problème au moindre carré qui est comparée dans ces tests. Les résultats obtenus sont présentés ci-dessous.

Average error on residual for `ch5-5-b1` :

```

- bunch_parlett : 0.0
- bunch_parlett! : 0.0
- bunch_kaufman : 2.934989140612743e-11
- bunch_kaufman! : 2.934989140612743e-11
- bunch_kaufman_rook : 2.934989140612743e-11

```

```

- bunch_kaufman_rook! : 2.934989140612743e-11
- bunch_kaufman de LAPACK : 0.0
- bunch_kaufman! de LAPACK : 0.0
Average error on residual for ch4-4-b1 :
- bunch_parlett : 0.0
- bunch_parlett! : 0.0
- bunch_kaufman : 1.8381054189831047e-9
- bunch_kaufman! : 1.8381054189831047e-9
- bunch_kaufman_rook : 1.8381054189831047e-9
- bunch_kaufman_rook! : 1.8381054189831047e-9
- bunch_kaufman de LAPACK : 0.0
- bunch_kaufman! de LAPACK : 0.0
Average error on residual for ch4-4-b2 :
- bunch_parlett : 0.0
- bunch_parlett! : 0.0
- bunch_kaufman : 0.0
- bunch_kaufman! : 0.0
- bunch_kaufman_rook : 0.0
- bunch_kaufman_rook! : 0.0
- bunch_kaufman de LAPACK : 0.0
- bunch_kaufman! de LAPACK : 0.0
Average error on residual for n3c5-b2 :
- bunch_parlett : 0.0
- bunch_parlett! : 0.0
- bunch_kaufman : 0.0
- bunch_kaufman! : 0.0
- bunch_kaufman_rook : 0.0
- bunch_kaufman_rook! : 0.0
- bunch_kaufman de LAPACK : 0.0
- bunch_kaufman! de LAPACK : 0.0

```

5 Discussion

5.1 Implémentations

D'un point de vue théorique, un théorème permet d'assurer la stabilité du processus de Bunch-Kaufman. Ce théorème assure que pour $(A + \Delta A)\hat{x} = b$ la solution du système $\hat{x}$, la perturbation sur la norme de A est bornée supérieurement comme $\|\Delta A\|_M \leq p(n)\rho_n u \|A\|_M + O(u^2)$ où p est quadratique. Ce résultat s'applique si le système est résolu par élimination gaussienne ou en utilisant l'inverse. Aucun résultat semblable n'est associé à Bunch-Parlett.

Pour tous les processus d'intérêt dans ce projet, un autre résultat intéressant est le suivant:

$$\frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} \leq p(n)u \operatorname{cond}(A) \operatorname{cond}(|D||L^T|) + O(u^2)$$

$\operatorname{cond}(|D||L^T|)$ n'est pas borné pour le pivotage partiel, mais il l'est pour le *rook pivoting* et cette méthode est donc en théorie moins sensible à un A initial moins bien conditionné.

D'un point de vue expérimental, on observe d'abord que toutes les implémentations de ces factorisations permettent de reconstruire la matrice A en entrée avec une précision d'au moins $1e^{-8}$ et ce, pour divers types de données. Qui plus est, les matrices L et B sont respectivement triangulaire inférieure et hermitienne tridiagonale. Les implémentations semblent donc valides.

Ensuite, le `Benchmarking` indique qu'elles effectuent la factorisation avec moins d'allocations que les versions standard de Julia qui appellent LAPACK. Elles semblent donc très bien optimisées par rapport aux allocations. De plus, pour chaque algorithme de factorisation, la version `!` génère bel et bien moins d'allocations que la version avec une `struct`, ce qui est attendu. Dans le contexte présent, on note qu'il est impossible de n'avoir aucune allocation, car il y a toujours quelques objets dont la création est nécessaire (vecteur permutation, vecteur de position des blocs 2×2 , etc.). Qui plus est, le reste des allocations n'ayant pas été éliminées semblent être générées lors de certaines exécutions d'opérations `==` pourtant effectuées éléments par éléments dans des boucles `for`. Ces opérations ne semblent pas pouvoir être optimisées davantage et il est assez surprenant qu'elles soient parfois source d'allocations. Peut-être est-ce lié à ce que les fonctions de `BenchmarkTools` considèrent comme une allocation et à la manière dont elles les détectent et les identifient.

Par ailleurs, on remarque que les implémentations sont pratiquement aussi rapides que celles de Julia appelant LAPACK pour des matrices 75×75 , donc relativement de petite taille. Puisque LAPACK est en Fortran et est ainsi optimisé pour la performance, il était attendu que les implémentations standard en Julia soit plus rapides. Toutefois, il est intéressant de constater que la différence de vitesse n'est pas si significative pour une telle taille matricielle. Bien qu'il reste à tester cela sur des matrices de plus grande taille, ceci indique que les implémentations sont bien optimisées en performance, surtout considérant qu'elles utilisent moins d'allocations, ce qui est d'ailleurs un facteur qui peut contribuer à améliorer le temps d'exécution. En outre, on remarque que Bunch-Parlett est plus lent que Bunch-Kaufman avec *Rook Pivoting*, qui est plus lent que Bunch-Kaufman classique. Ceci est attendu, car Bunch-Parlett utilise une vérification plus exhaustive des valeurs absolues maximales au sein de la matrice et possède des opérations de double permutation au sein d'une même itération. En comparaison, Bunch-Kaufman avec *Rook Pivoting* possède aussi des opérations de double permutation, mais utilise la vérification allégée des valeurs absolues maximales propre à Bunch-Kaufman. Bunch-Kaufman classique possède aussi cette vérification allégée, mais n'a en plus que des opérations de permutation simple.

Pour la résolution de problèmes aux moindres carrés aléatoires, l'erreur comparativement à celle trouvée par factorisation `qr` est d'abord extrêmement faible pour toutes les implémentations. Ceci indique leur validité dans la résolution de système linéaire. On remarque même que l'erreur de l'implémentation de Bunch-Kaufman est inférieure à celle obtenue avec l'implémentation de LAPACK, ce qui suggère une plus grande précision. Bunch-Parlett et Bunch-Kaufman avec *Rook Pivoting* présentent une plus grande erreur, bien que la différence demeure minime en `Big Float`. Peut-être Bunch-Kaufman classique est-il plus efficace sur des problèmes aléatoires et ainsi probablement bien conditionnés. De plus, pour chaque implémentation, les versions `struct` et `!` ont la même erreur, y compris pour l'implémentation LAPACK. Ceci est attendu puisque la factorisation est la même dans les deux cas.

Pour les problèmes aux moindres carrés issus de `MatrixMarket`, l'erreur moyenne sur le résidu est tellement faible qu'elle tombe à 0 presque tout le temps. Les seules exceptions concernent Bunch-Kaufman avec ou sans *rook pivoting* pour `ch5-5-b1` et `ch4-4-b1`, bien que les erreurs soient quand même extrêmement faibles. Il est difficile d'identifier précisément pourquoi; les deux premiers problèmes sont ceux créés à partir des matrices aux nombres de conditionnement les plus bas, mais aux normes les plus élevées. Le noyau des matrices A de ces deux problèmes est aussi de taille 1, ce qui n'est pas le cas des deux derniers problèmes, pour lesquels la dimension du noyau est plus élevée. Sur ces deux premiers problèmes, on observe également que Bunch-Parlett est plus précis que Bunch-Kaufman, ce qui peut s'expliquer étant donné la vérification plus exhaustive des valeurs absolues maximales qu'utilise Bunch-Parlett. Tout ceci renforce la validité dans la résolution de systèmes linéaires.

5.2 Difficultés rencontrées

Trois difficultés principales ont été rencontrées:

1. Afin d'effectuer la factorisation en chaîne sur les différents compléments de Schur sujets à des permutations, et de ramener cesdites permutations dans un seul opérateur de permutation qui s'applique à l'ensemble du bloc LBL' , il est nécessaire de permuter les lignes de L de façon appropriée au cours de la factorisation. Cela n'est pas nécessairement évident à première vue, et n'est pas

toujours explicité dans les sources bibliographiques. Ceci se fait explicitement dans la version `struct`, et se fait en permutant directement les éléments des colonnes antérieures de A dans la version `!`.

2. Afin d'effectuer des permutations de lignes et de colonnes dans une matrice hermitienne dans laquelle seul le triangle inférieur est stocké, il est nécessaire d'adapter l'opération de permutation de façon séquentielle, en permutant parfois une partie de ligne avec une partie de colonne. En effet, puisque le triangle supérieur est le transposé conjugué du triangle inférieur, ses colonnes sont en fait les lignes du triangle inférieur et inversement. Ceci est effectué élément par élément sans allocation grâce à la fonction `perm_r1_et_r2!`.

```
function perm_r1_et_r2!(M, r1::Int, r2::Int)
    """
    Modifie une matrice M hermitienne en permutant les lignes et les colonnes d'indices r1 et r2, où
    ↪ r2 >= r1
        Entrée : - Matrice M hermitienne
                - Indice r1
                - Indice r2
    """
    if r2 < r1
        error("r2 doit être plus grand ou égal à r1")
    end

    n = size(M, 1)

    M[r1, r1], M[r2, r2] = M[r2, r2], M[r1, r1]
    M[r2, r1] = conj(M[r2, r1])
    for k in r1+1:r2-1
        M[k, r1], M[r2, k] = conj(M[r2, k]), conj(M[k, r1])
    end
    for i in r2+1:n
        M[i, r1], M[i, r2] = M[i, r2], M[i, r1]
    end
    for j in 1:r1-1
        M[r1, j], M[r2, j] = M[r2, j], M[r1, j]
    end
end
```

3. Afin d'optimiser le nombre d'allocations, il a fallu investiguer précisément chaque portion de code afin d'identifier ce qui en générerait. Ceci semble avoir révélé que parfois, de simples opérations comme `.*=` appliquées sur une *view*, ou encore un calcul dans lequel se retrouve à la fois des variables et des fonctions comme `abs` ou `conj` appliquées à des variables, sont sources d'allocations. Le tout a donc dû être optimisé au peigne fin, avec beaucoup de recours aux boucles `for` et à la création de variables intermédiaires.

6 Retour sur les objectifs

La majorité des objectifs du projet sont accomplis avec la remise de la phase 2.

1. Comprendre et documenter les factorisations de Bunch-Parlett et Bunch-Kaufman (pivotage complet et pivotage partiel.) Fournir des exemples, des schémas explicatifs, faire des liens avec les éléments vus en classe.
 - Voir la section Section 1 et la section Section 2.
2. Implémenter ces deux factorisation ainsi que l'alternative du *rook pivoting*. Les comparer et identifier leurs avantages et leurs inconvénients respectifs.

- Voir la section Section 3 et la section Section 5.
 - Le nombre de flop lui-même n'a pas été comparé directement, d'un point de vue numérique.
 - La sous-section Section 3.2 explique comment les fonctions ont été optimisées pour réduire le temps d'exécution ainsi que le nombre d'allocation.
3. Comparer nos implémentations à celles déjà disponibles dans Julia et tenter d'identifier les sources possibles des différences observées.
 - Voir la section Section 4.
 - Il a été confirmé qu'aucune implémentation de Bunch-Parlett n'existe dans Julia.
 - L'option *rook pivoting* devait être testée, mais ne fonctionnait pas lors de nos tests.
 4. Joindre la factorisation à des méthodes de résolution de systèmes linéaires.
 - Voir la section Section 4, plus spécifiquement la redéfinition de la fonction $\backslash$.
 5. Appliquer l'implémentation à un ensemble de problèmes modèles ainsi qu'à un problème plus spécifique au choix.
 - Voir la section Section 4.
 - Différents problèmes ont été créés avec des matrices aléatoires ainsi qu'avec des matrices connues et définies.

Tel qu'attendu, la majorité du temps a été alloué à l'optimisation des implémentations en Julia et à la revue de littérature. Cette dernière a couvert à la fois la théorie derrière la factorisation et les normes de codage en Julia. Le fait de pouvoir réutiliser des portions de code pour chaque implémentation ainsi que pour les tests a permis de réduire un peu le temps passé sur ces éléments.

7 Références

[1] : Nicholas J. Higham. Accuracy and Stability of Numerical Algorithms. Society for Industrial and Applied Mathematics, second edition, 2002. doi: 10.1137/1.9780898718027. URL <https://epubs.siam.org/doi/abs/10.1137/1.9780898718027>

[Lien vers le dépôt Github du projet](#)